Proyecto #2 Arquitectura de Computadores y Ensambladores

Angel Rafael Juárez Higueros, 202501037

Facultad de Ingeniería, Universidad de San Carlos de Guatemala

Ing. Otto Rene Escobar Leiva

Aux. Diego Josue Guevara Abaj

23 de octubre del 2026

1. Enfoque del Desarrollo

Para armar este shell en ensamblador AArch64 sin usar C, la principal meta fue mantener todo lo más sencillo posible en cuanto a manejo de memoria. En lugar de complicarme pidiendo pedazos de memoria dinámica cada vez que alguien escribía un comando, decidí usar memoria que ya estaba reservada desde el principio.

Cuando el usuario escribe algo, se guarda en un espacio de 128 bytes. Luego, mi programa recorre ese texto y va cambiando los espacios en blanco por caracteres nulos (ceros). Al mismo tiempo que hace eso, va guardando en qué posición empieza cada palabra dentro de un arreglo. Esto es clave porque el sistema operativo necesita que los argumentos estén acomodados en un bloque seguido para poder ejecutarlos, y esta forma de hacerlo cumple justo con eso sin gastar memoria de más.

Para saber qué comando ejecutar, el shell toma la primera palabra que encontró y la compara con los comandos que hice a mano (help, clear, echo, exit). Si hace match con alguno, salta a esa parte del código. Si no encuentra nada parecido, asume que es un comando del sistema como ls o pwd y trata de correrlo. Como un pequeño detalle para que sea más fácil de usar, si pones un comando corto sin la ruta completa, el programa le pega automáticamente "/bin/" al inicio para ayudar a encontrarlo rápido.

2. Manejo de la Pila (Stack)

Trabajar con la pila en ensamblador es engañoso, porque si se te olvida guardar algo importante, el programa se cierra de golpe. Trate de seguir la regla de que la pila siempre tiene que crecer o encogerse en múltiplos de 16 bytes, porque así lo pide el procesador.

Cuando una función tiene que llamar a otra función adentro, uso la instrucción STP para guardar un par de cosas en la pila: el registro x19 (que uso mucho para no perder la pista de dónde están los argumentos) y el registro x30 (que es la dirección a donde tengo que regresar cuando termine). Cuando la función ya va a terminar, uso LDP para sacar esas cosas de la pila y dejar todo como estaba.

Ahora, para no hacer trabajar de más a la computadora, si tengo una función corta que solo hace comparaciones o copias y no llama a nadie más, no uso la pila para nada. Es una forma de mantener el código rápido.

3. Llamadas al Sistema Operativo

Como no podíamos usar librerías externas, todo lo que hace el shell por debajo se comunica directo con Linux mandando el número de la acción al registro x8 y llamando a la interrupción svc #0. Estas fueron las principales que usé:

- Para mostrar cosas en la consola, usé la número 64 (write). Solo le digo que escriba en la salida estándar y le paso el texto.
- Para leer lo que teclea el usuario, usé la 63 (read).
- Para los comandos como ls o date, usé la 220 (clone). Esta crea un proceso hijo, así el shell no se muere cuando corre otro programa.
- Dentro de ese proceso hijo, usé la 221 (execve) para reemplazar al hijo por el programa que el usuario pidió.
- Y mientras el hijo hace su trabajo, el proceso padre (mi shell) se queda esperando con la syscall 260 (wait4) para no volver a tirar la línea de comando antes de tiempo.

En general, el proyecto se armó como un rompecabezas, conectando la lectura, separando el texto y delegando las tareas al sistema.